

Week 05

Responsive Design

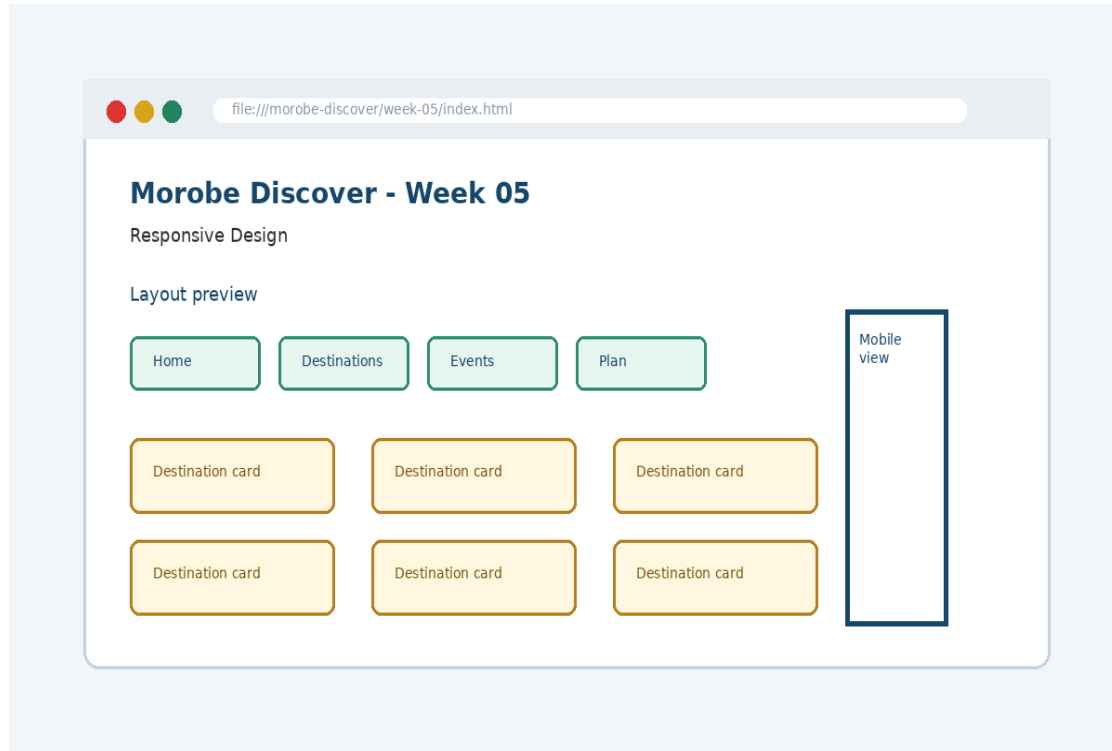
Adapt the website interface to function across various device screen sizes.



Learning outcomes

Adapt the website interface to function across various device screen sizes.
Explain the core programming/design concept for this week
Read and modify key code patterns
Test the weekly project increment
Prepare evidence for GitHub submission

Today's problem



Why it matters

Start with flexible layouts and add breakpoints when the layout needs help, not because of a device name.

Project before -> after

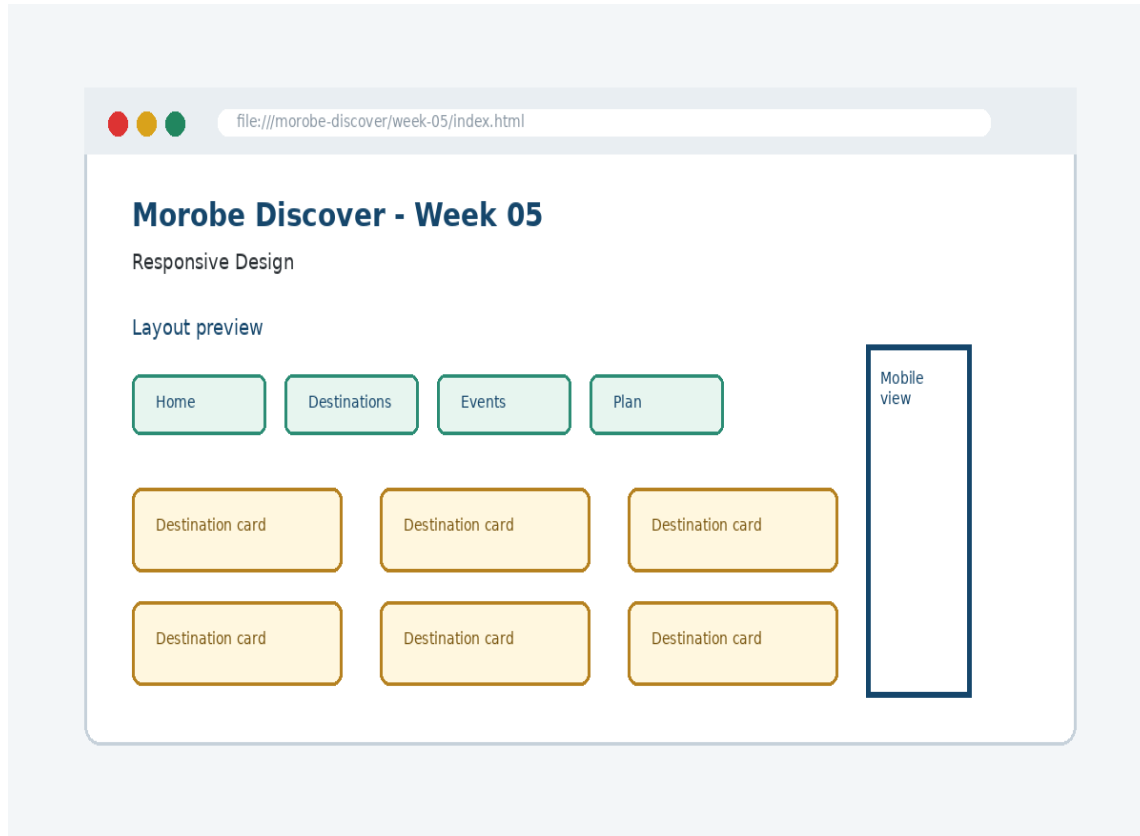
Before Week 05

Week 04 created structured layouts using Flexbox and Grid.

After Week 05

The same site works on mobile, tablet and desktop without separate versions.

What we build this week



Stack navigation and cards on small screens
Add a tablet/desktop breakpoint
Make images and forms fluid
Test the main user journey at several widths
Record screenshots as evidence

Core principle

Start with flexible layouts and add breakpoints when the layout needs help, not because of a device name.

Keep in mind

A good web developer can point to the code, the browser result, and the user reason behind the decision.

Key concepts

Concept 1

Mobile-first CSS defines the simplest layout first

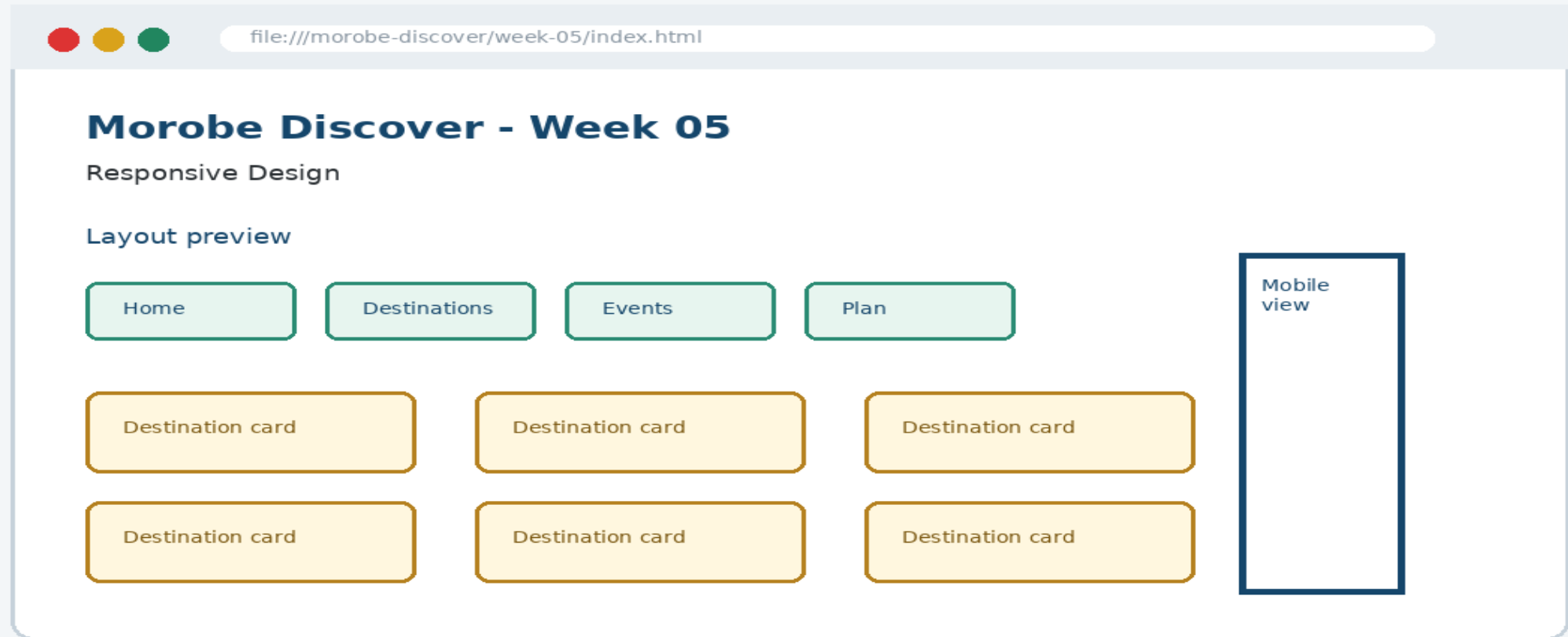
Concept 2

Media queries add layout changes at meaningful widths

Concept 3

Images and grids should be fluid

Project anatomy



Key code pattern A

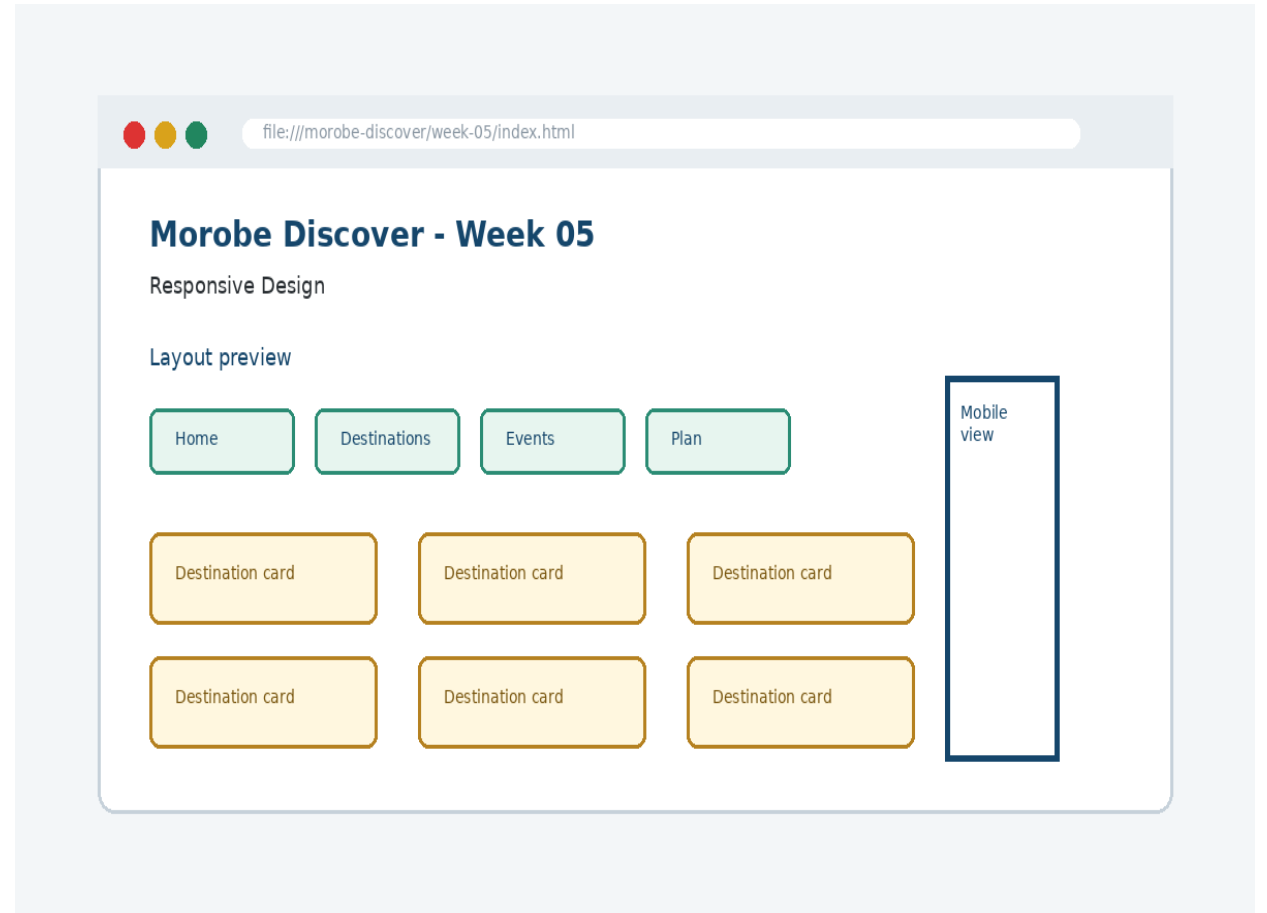
```
img {  
  max-width: 100%;  
  height: auto;  
}
```

Explain it

Which file contains this code?
Which line creates the visible change?
What should appear in the browser?

Code -> visual result

```
img {  
  max-width: 100%;  
  height: auto;  
}
```



What changed?

Project file changed for Responsive Design

The browser output is now closer to the required weekly evidence

The change supports a user task or quality requirement

The change can be tested and committed

Key code pattern B

```
@media (min-width: 768px) {  
  .card-grid {  
    grid-template-columns: repeat(3, 1fr);  
  }  
}
```

Why this pattern is needed

The second pattern usually applies the same idea to a different component, state, layout or data flow.

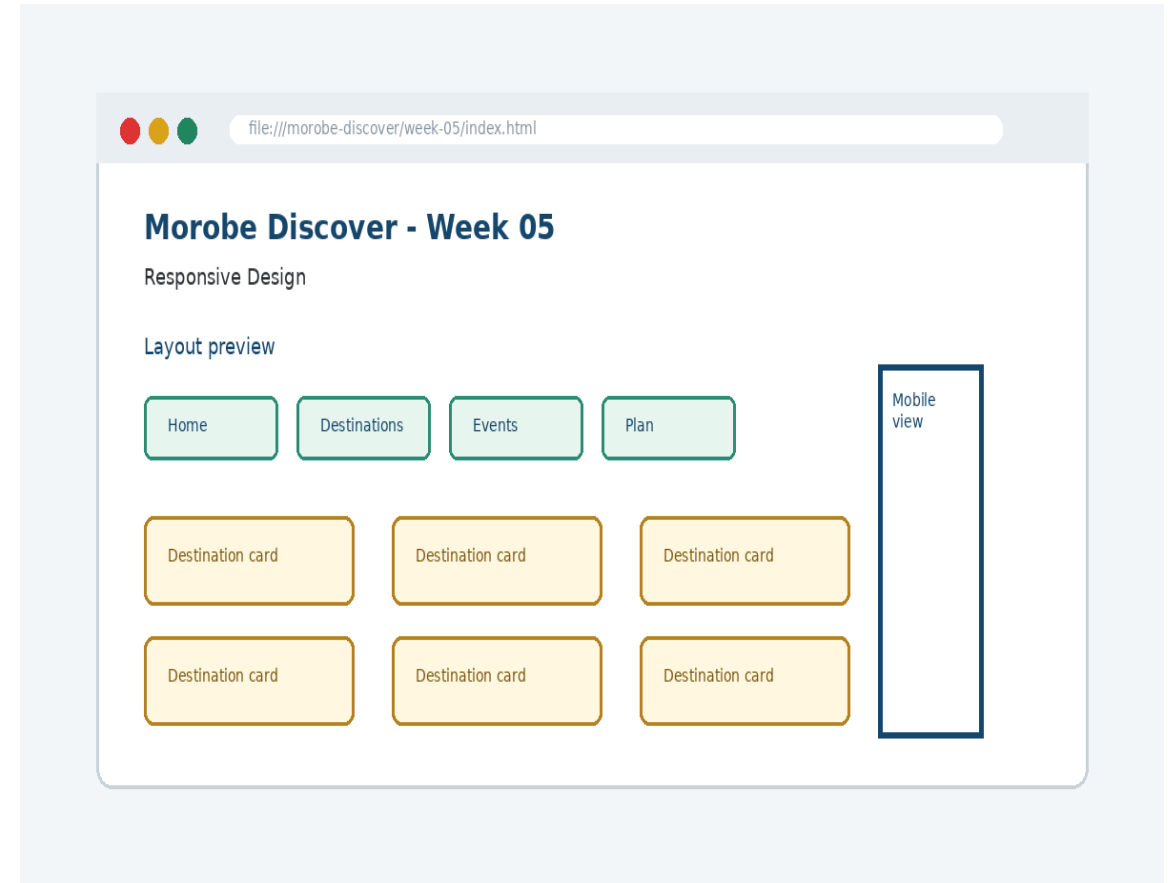
Apply it to Morobe Discover

Stack navigation and cards on small screens

Add a tablet/desktop breakpoint

Make images and forms fluid

Test the main user journey at several widths



Compare the approaches

Weak approach

Memorise the definition
Copy code without understanding
No browser evidence
No test note

Professional approach

Explain the concept
Apply the correct code pattern
Inspect the result
Commit and describe the evidence

Common mistake

Mistake

The page appears unchanged, but the student assumes the feature is complete.

Correction

Check the correct file, refresh the browser, inspect DevTools, and verify the expected output.

Why does this fail?

```
// Broken checkpoint  
// File changed, but the browser result does not  
change.  
// Possible causes:  
// - wrong file  
// - file not linked  
// - syntax error  
// - browser not refreshed
```

Class question

Which cause would you check first, and why?

Fix the problem

- Confirm the browser is opening the correct file
- Check links to CSS/JS/data files
- Inspect the console for errors
- Use a small working example
- Test again before committing

Class activity

Activity - 7 minutes

Find the first browser width where the card layout begins to fail.

Work in pairs

Point to the project evidence

Prepare one answer to share

Implementation steps 1-3

Stack navigation and cards on small screens
Add a tablet/desktop breakpoint
Make images and forms fluid

Implementation steps 4-5

Test the main user journey at several widths
Record screenshots as evidence

LIVE DEMO

Demo checklist

1. Open the current project
2. Locate the affected file
3. Add or modify the key code
4. Refresh the browser
5. Inspect the result
6. Explain the change



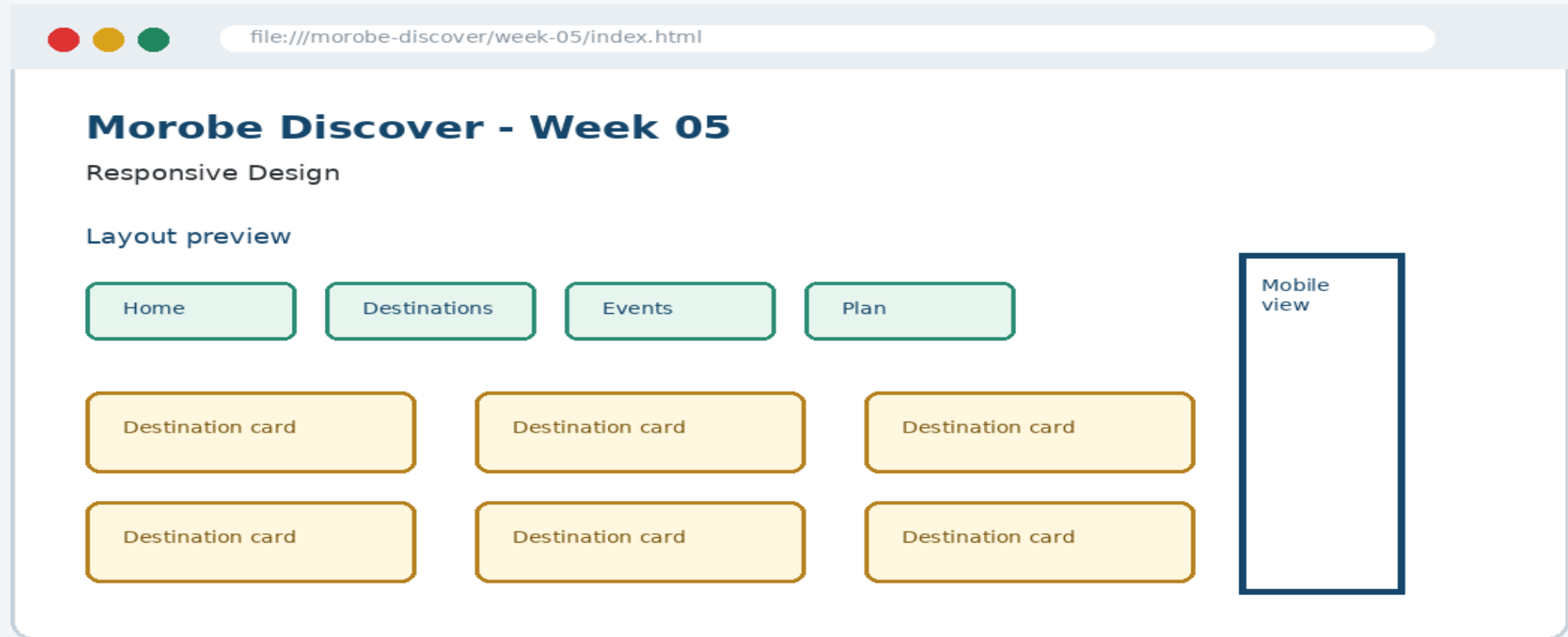
Predict the output

```
@media (min-width: 768px) {  
  .card-grid {  
    grid-template-columns: repeat(3, 1fr);  
  }  
}
```

Prediction prompt

What should change on screen when this code is applied correctly?

Browser result



Testing the feature

Use DevTools responsive mode
Test 360px, 768px and desktop widths
Check tap targets and line length
Confirm no horizontal scrolling

Inspect with browser tools

Inspection path

Right click -> Inspect
Elements panel -> structure/classes
Console -> JavaScript errors
Network/Application -> data/storage when relevant

Evidence to save

Screenshot
Short test note
Changed files
Git commit message

Git evidence

```
git add .  
git commit -m "Week 05 Responsive Design project  
increment"
```

Why it matters

Git history shows progression from Week 01 to final deployment. It is part of the assessment evidence.

Complete source code

Repository:
`https://github.com/YOUR-USERNAME/is229-morobe-discover`

Weekly folder:
`/week-05/`

Complete project:
`/complete-project/`

Reminder

Do not paste a whole file into a slide. Use the repository for complete code.

Mini challenge

Challenge

Modify one part of the Week 05 implementation and explain what changed in the browser.

State the file edited

State the line/section changed

Show the before and after result

What counts as evidence?

Technical evidence

Changed code files
Browser screenshot
Test note
Git commit

Understanding evidence

Code explanation
Why this method fits
Connection to previous week
Connection to next week

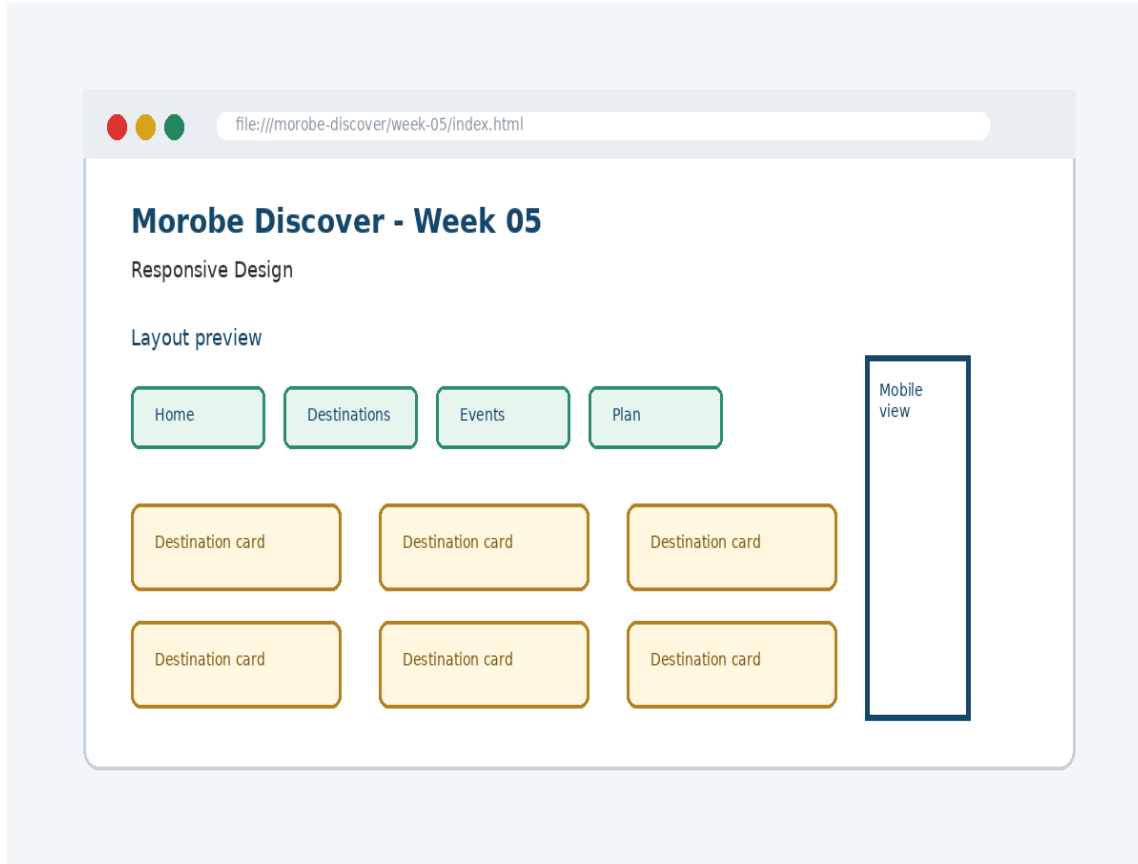
Project lab handoff

- Stack navigation and cards on small screens
- Add a tablet/desktop breakpoint
- Make images and forms fluid
- Test the main user journey at several widths
- Record screenshots as evidence

Definition of done

Week 05 feature is implemented
Project still preserves earlier week features
Browser result matches the expected output
Test note and screenshot are saved
Git commit message is meaningful

What we built today



The same site works on mobile, tablet and desktop without separate versions.
Evidence: Responsive rules, media queries and fluid mobile-friendly layouts
Technology: CSS Media Queries

Lecture summary

Start with flexible layouts and add breakpoints when the layout needs help, not because of a device name.

Mobile-first CSS defines the simplest layout first

Media queries add layout changes at meaningful widths

Testing and Git are part of development, not extras

Prepare for next class

Finish this week's lab before the next class
Read the next topic in advance
Bring the current project ready to extend
Be ready to explain one code decision

Exit question

How did Week 05 improve Morobe Discover, and what will the next week build on?